ESE 124 - Exercise 10

Mealy State Machines with switch and 2D Arrays

Fall 2026 | Introductory level | Individual work + peer learning | 100 points | about 3 hours

What you do in this exercise

There are 5 questions. Each one is a small state machine. In every folder main.c, the .h file and the Makefile are already written for you. You only open ONE .c file and finish its TODOs.

Q	Folder	Open and edit	Way	Points
Q1	q1_vending	vending.c	2D array	15
Q2	q2_frfr	frfr.c	switch	15
Q3	q3_brobank	atm.c (only atm_step)	switch	20
Q4	q4_lock	lock.c	2D array	20
Q5	q5_comments	strip.c	switch AND 2D array	25
	reflection.txt	your answers		5

How every question works (the same 5 steps)

- **1.** Look at the diagram and the table in this handout.
- **2.** Open the .c file and do TODO 1, TODO 2, TODO 3 ... in order.
- **3.** Save. In the terminal: make run (inside the question folder).
- **4.** Compare what you see with expected_output.txt in the same folder.
- **5.** Run the autograder for that question and fix what it says.

The idea: a Mealy machine

A state machine is always in ONE state. Each input follows ONE arrow. Every arrow is written input/output: 1/0 means "read 1, give output 0". All 5 questions are Mealy machines and use the same loop:

Step	Code	Why
1. read the input	col = column_of(c);	turn the char into a column number
2. remember the state	old = s;	you print it, and it decides the output
3. output	out = out_table[s][col];	the output belongs to the ARROW
4. move LAST	s = next_state[s][col];	follow the arrow to the next state

You will write the arrows in two ways:

```
Way 1 - switch:    switch (state) { case A: if (input == ...) { ... } break; ... }
Way 2 - 2D array:  out = out_table[state][input];   state = next_state[state][input];
```

The cheat sheet at the end of this handout shows ONE machine written both ways.

How to use the Makefile

Every folder has a Makefile. It remembers the long gcc command for you. Open a terminal in VS Code (Terminal > New Terminal) and type:

Type this	What happens
cd q1_vending	go into the Q1 folder (cd .. goes back up)
make	compiles main.c + vending.c into the program vending
make run	compiles if needed, then runs ./vending
make clean	deletes the program - your .c, .h and .txt files stay

Windows: if make is not found, type mingw32-make instead. This is the given Makefile in q1_vending (you do not change it):

```
CC      = gcc
CFLAGS = -Wall -Wextra -std=c11

vending: main.c vending.c vending.h
       $(CC) $(CFLAGS) main.c vending.c -o vending

run: vending
       ./vending

clean:
       rm -f vending vending.exe

.PHONY: run clean
```

Line	Meaning
CC = gcc CFLAGS = ...	the compiler and its options (-Wall -Wextra = show all warnings)
vending: main.c vending.c vending.h	a RULE: to make vending you need these 3 files
<TAB>$(CC) $(CFLAGS) ... -o vending	the command for the rule - it MUST start with a TAB
run: / clean:	the targets for make run and make clean

make says	It means
'vending' is up to date.	nothing changed since the last build - just type make run
missing separator	a command line starts with spaces instead of a TAB
make: command not found	Windows: type mingw32-make (or use the CAD Lab computers)
warning: ... / error: ...	read the FIRST one: it names the file and the line number

How to read the autograder

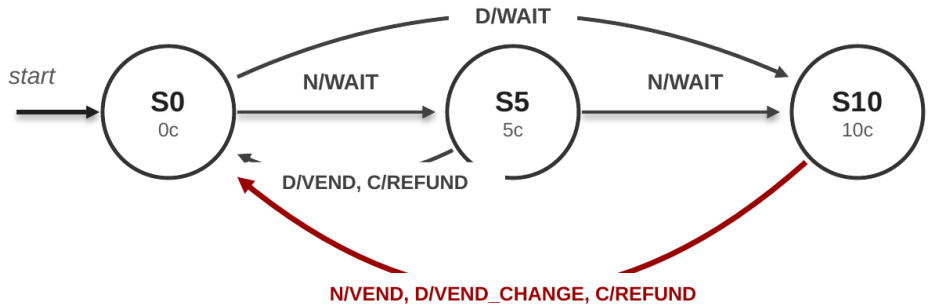
Run it in the main folder: py tests\run_all_tests.py (Windows, or double-click run_tests.bat) or python3 tests/run_all_tests.py (Mac). Add q2 to test only Q2. Each FAIL shows the input, every expected line marked [OK] or [FAIL], what your program printed, and a HINT. Upper/lower case and extra spaces do not matter.

Important: The way is checked. A switch function must use switch. A table function must use only the tables (no switch, no if) - the test changes one cell of your table to check that your function reads it.

Question 1 - BroBot's 15-cent Vending Machine (15 points)

Folder	Open and edit	Way	Program
q1_vending	vending.c	2D array	vending

A snack costs 15 cents. The state is the money inside the machine: S0, S5 or S10. The inputs are N = nickel (5 c), D = dime (10 c), Q = quarter (not accepted) and C = cancel.



Not drawn: Q/REJECT on every state (stay) and C/WAIT on S0. 12 arrows = 12 table cells.

The table - each cell is next state / output:

State	N (NICKEL)	D (DIME)	Q (QUARTER)	C (CANCEL)
S0	S5 / WAIT	S10 / WAIT	S0 / REJECT	S0 / WAIT
S5	S10 / WAIT	S0 / VEND	S5 / REJECT	S0 / REFUND
S10	S0 / VEND	S0 / VEND_CHANGE	S10 / REJECT	S0 / REFUND

Do these steps:

- **Step 1:** TODO 1 - copy the LEFT part of every cell into next_state and the RIGHT part into action_table. Example: row S5, column D -> next_state[S5][DIME] = S0 and action_table[S5][DIME] = VEND.
- **Step 2:** TODO 2 - column_of(c): return NICKEL for 'N', DIME for 'D', QUARTER for 'Q', CANCEL for 'C', and -1 for any other char.
- **Step 3:** TODO 3 - vend_step: three lines, no switch and no if: Action a = action_table[*s][col]; *s = next_state[*s][col]; return a;
- **Step 4:** in the terminal: cd q1_vending then make run - compare with expected_output.txt.
- **Step 5:** in the main folder: py tests\run_all_tests.py q1 (Mac: python3 tests/run_all_tests.py q1).

Input file coins.txt:

```
N D
D D
Q N C
N N N
D x N
D
```

Expected output:

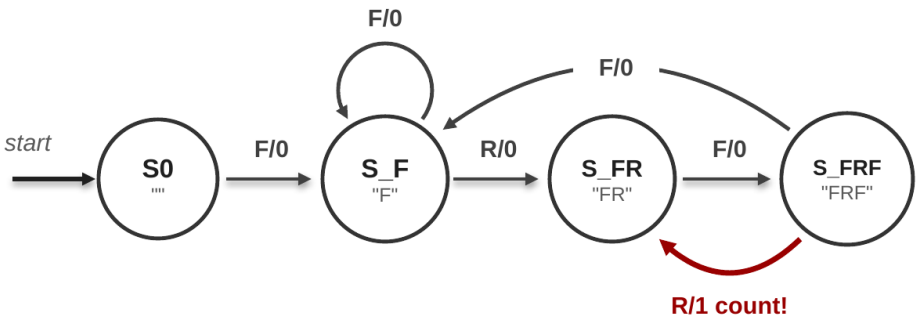
```
STEP 1: S0 --N/WAIT--> S5
STEP 2: S5 --D/VEND--> S0
STEP 3: S0 --D/WAIT--> S10
STEP 4: S10 --D/VEND_CHANGE--> S0
STEP 5: S0 --Q/REJECT--> S0
STEP 6: S0 --N/WAIT--> S5
STEP 7: S5 --C/REFUND--> S0
```

```
STEP 8: S0 --N/WAIT--> S5
STEP 9: S5 --N/WAIT--> S10
STEP 10: S10 --N/VEND--> S0
STEP 11: S0 --D/WAIT--> S10
STEP 12: S10 --N/VEND--> S0
STEP 13: S0 --D/WAIT--> S10
VENDS=4
CHANGE_CENTS=5
REFUND_CENTS=5
FINAL=S10
```

Question 2 - frfr - Count a Pattern (15 points)

Folder	Open and edit	Way	Program
q2_frfr	frfr.c	switch	frfr

Count how many times "frfr" (for real, for real) appears in each chat message. 'f' and 'F' are the same. Matches may OVERLAP: frfrfr contains 2. The state = how much of F R F R you have read so far. The output 1 means "one more frfr found".



Every other letter -> S0 (output 0). 'f' = 'F'. The match goes to S_FR: FRFRFR has 2.

The table - each cell is next state / output ("other" = any other char, also a space):

State	F	R	other
S0	S_F / 0	S0 / 0	S0 / 0
S_F	S_F / 0	S_FR / 0	S0 / 0
S_FR	S_FRF / 0	S0 / 0	S0 / 0
S_FRF	S_F / 0	S_FR / 1 (found!)	S0 / 0

Do these steps:

- **Step 1:** TODO 1 - frfr_step: first char u = (char)toupper((unsigned char)c); *found = 0; then switch (s) with one case per state (row of the table). Inside a case use if (u == 'F') / if (u == 'R') and return the next state. Only in S_FRF with R: set *found = 1 and return S_FR.
- **Step 2:** TODO 2 - count_frfr: if text is NULL return 0. Start with s = S0 and count = 0; for every char call s = frfr_step(s, text[i], &found); and add found to count. Return count.
- **Step 3:** in the terminal: cd q2_frfr then make run - compare with expected_output.txt.
- **Step 4:** in the main folder: py tests\run_all_tests.py q2 (Mac: python3 tests/run_all_tests.py q2).

Input file messages.txt:

```
that lab was hard frfr
frfrfr no cap

BroBot said FRFRFRFR
for real for real
FrFr fRfR
frffrfr
```

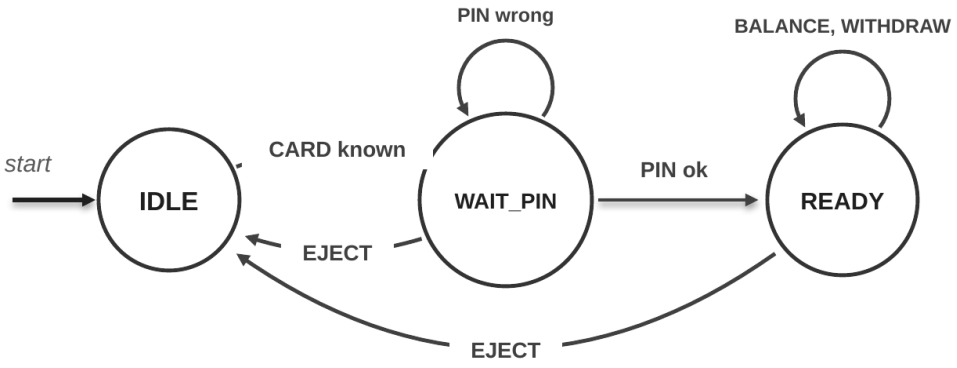
Expected output:

```
MSG 1: FRFR=1
MSG 2: FRFR=2
MSG 3: FRFR=3
MSG 4: FRFR=0
MSG 5: FRFR=2
MSG 6: FRFR=1
MESSAGES=6
TOTAL=9
MOST=3
```

Question 3 - BroBank W10: The ATM (20 points)

Folder	Open and edit	Way	Program
q3_brobank	atm.c (only atm_step)	switch	atm

The same line means different things in different states: "PIN 1234" is REJECTED when no card is inside, but logs you in while the ATM waits for a PIN. The output of each arrow is a message.



Every line that does not fit the state: REJECTED, the state stays the same.

The table - what each line does in each state:

State	Line	Condition	Next state	Message (output)
IDLE	CARD name	name is known	WAIT_PIN	INSERT PIN
IDLE	CARD name	name is unknown	IDLE	NO SUCH ACCOUNT
WAIT_PIN	PIN p	p is correct	READY	PIN OK
WAIT_PIN	PIN p	p is wrong	WAIT_PIN	WRONG PIN
WAIT_PIN	EJECT		IDLE	CARD RETURNED
READY	BALANCE		READY	BALANCE 500.00
READY	WITHDRAW a	a <= 0	READY	INVALID AMOUNT
READY	WITHDRAW a	a > balance	READY	INSUFFICIENT FUNDS
READY	WITHDRAW a	otherwise (balance - a)	READY	CASH 120.00
READY	EJECT		IDLE	CARD RETURNED

any	anything else		same	REJECTED

Do these steps:

- **Step 1:** Read the given first lines of atm_step: cmd = the first word, arg = the second, got = how many words. msg is already "REJECTED" - so any line that matches no row keeps that message.
- **Step 2:** Write switch (m->state) { case IDLE: ... break; case WAIT_PIN: ... break; case READY: ... break; default: break; }
- **Step 3:** Inside each case, check the rows of that state with if / else if and strcmp(cmd, "CARD") == 0. Set the next state (m->state = ...) and the message (strcpy(msg, "PIN OK")).
- **Step 4:** Numbers: sscanf(arg, "%d", &pin) == 1 for a PIN, sscanf(arg, "%lf", &amt) == 1 for an amount. Money: sprintf(msg, "CASH %.2f", amt).
- **Step 5:** in the terminal: cd q3_brobank then make run - compare with expected_output.txt.
- **Step 6:** in the main folder: py tests\run_all_tests.py q3 (Mac: python3 tests/run_all_tests.py q3).

Input files accounts.txt (left) and session.txt (right):

```
3                       PIN 1234
Alice 1234 500.00       CARD Alice
Bro 2024 80.00          PIN 1111
Chen 1111 0.00          PIN 1234
                        BALANCE
                        WITHDRAW 120
                        WITHDRAW 900
                        EJECT
                        CARD Dave
                        CARD Bro
                        BALANCE
                        PIN 2024
                        WITHDRAW 0
                        WITHDRAW 80
                        BALANCE
                        EJECT
                        EJECT
```

Expected output:

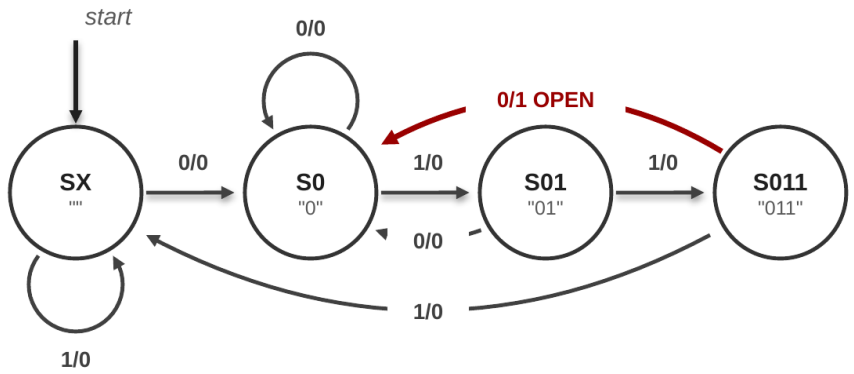
```
EVENT 1: PIN 1234 | IDLE -> IDLE | REJECTED
EVENT 2: CARD Alice | IDLE -> WAIT_PIN | INSERT PIN
EVENT 3: PIN 1111 | WAIT_PIN -> WAIT_PIN | WRONG PIN
EVENT 4: PIN 1234 | WAIT_PIN -> READY | PIN OK
EVENT 5: BALANCE | READY -> READY | BALANCE 500.00
EVENT 6: WITHDRAW 120 | READY -> READY | CASH 120.00
EVENT 7: WITHDRAW 900 | READY -> READY | INSUFFICIENT FUNDS
EVENT 8: EJECT | READY -> IDLE | CARD RETURNED
EVENT 9: CARD Dave | IDLE -> IDLE | NO SUCH ACCOUNT
EVENT 10: CARD Bro | IDLE -> WAIT_PIN | INSERT PIN
EVENT 11: BALANCE | WAIT_PIN -> WAIT_PIN | REJECTED
EVENT 12: PIN 2024 | WAIT_PIN -> READY | PIN OK
EVENT 13: WITHDRAW 0 | READY -> READY | INVALID AMOUNT
EVENT 14: WITHDRAW 80 | READY -> READY | CASH 80.00
EVENT 15: BALANCE | READY -> READY | BALANCE 0.00
EVENT 16: EJECT | READY -> IDLE | CARD RETURNED
EVENT 17: EJECT | IDLE -> IDLE | REJECTED
FINAL=IDLE
REJECTED=3
ACCOUNT Alice 380.00
```

```
ACCOUNT Bro 0.00
ACCOUNT Chen 0.00
```

Question 4 - The CAD Lab Door Lock (20 points)

Folder	Open and edit	Way	Program
q4_lock	lock.c	2D array	lock

Type 0 1 1 0 and the door opens. The state remembers how much of 0 1 1 0 you have typed: SX (nothing), S0, S01, S011. The output is 1 on the key that completes 0110 (OPEN), else 0. Any key that is not 0 or 1 is a wrong key: column ERR, back to SX. Every line of keypad.txt starts in SX.



Mealy: input/output on every arrow. Any wrong key (ERR) -> SX with output 0.

The table - each cell is next state / output:

State	0 (IN0)	1 (IN1)	other (ERR)
SX	S0 / 0	SX / 0	SX / 0
S0	S0 / 0	S01 / 0	SX / 0
S01	S0 / 0	S011 / 0	SX / 0
S011	S0 / 1 (OPEN)	SX / 0	SX / 0

Why S011 + 0 goes to S0: the last 0 you typed can be the first key of the next 0110 (overlap).

Do these steps:

- **Step 1:** TODO 1 - copy the LEFT part of every cell into next_state and the RIGHT part into out_table (4 rows x 3 columns). Example: next_state[S011][IN0] = S0 and out_table[S011][IN0] = 1.
- **Step 2:** TODO 2 - column_of(c): return IN0 for '0', IN1 for '1', and ERR for any other char.
- **Step 3:** TODO 3 - lock_step: three lines, no switch and no if: int out = out_table[*s][col]; *s = next_state[*s][col]; return out;
- **Step 4:** in the terminal: cd q4_lock then make run - compare with expected_output.txt.
- **Step 5:** in the main folder: py tests\run_all_tests.py q4 (Mac: python3 tests/run_all_tests.py q4).

Input file keypad.txt:

```
0110110
01100110
0111 0110

1101101100110
```

```
01#10110
1111
```

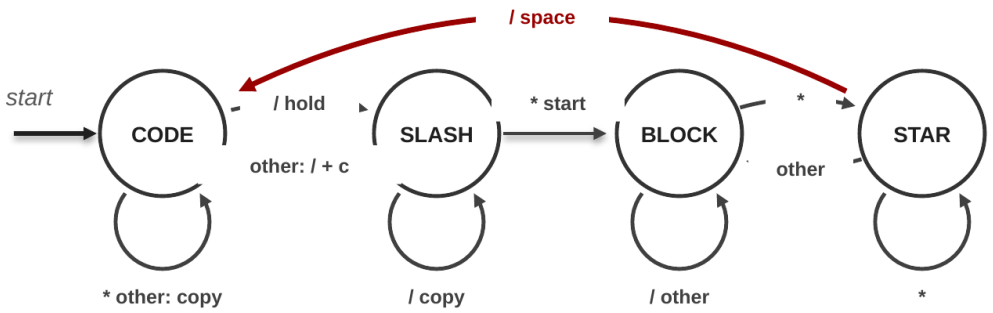
Expected output:

```
TRACE 1: SX S0 S01 S011 S0 S01 S011 S0
LOG 1: OUT=0001001 OPENS=2
TRACE 2: SX S0 S01 S011 S0 S0 S01 S011 S0
LOG 2: OUT=00010001 OPENS=2
TRACE 3: SX S0 S01 S011 SX SX S0 S01 S011 S0
LOG 3: OUT=000000001 OPENS=1
TRACE 4: SX SX SX S0 S01 S011 S0 S01 S011 S0 S0 S01 S011 S0
LOG 4: OUT=0000010010001 OPENS=3
TRACE 5: SX S0 S01 SX SX S0 S01 S011 S0
LOG 5: OUT=00000001 OPENS=1
TRACE 6: SX SX SX SX SX
LOG 6: OUT=0000 OPENS=0
LOGS=6
TOTAL_OPENS=9
```

Question 5 - Remove the Comments - one machine, two ways (25 points)

Folder	Open and edit	Way	Program
q5_comments	strip.c	switch AND 2D array	nocomment

The machine reads C code one char at a time and removes every comment that starts with slash-star and ends with star-slash. A '/' is held back (state SLASH) until the next char decides: '*' starts a comment, anything else means it was a division and the '/' is written back. A comment becomes ONE space. main.c runs your switch way AND your table way and prints AGREE=YES only if they give the same result.



4 states x 3 char classes = 12 arrows. A comment becomes ONE space.

The table - each cell is next state / output (what to write):

State	'/' (C_SLASH)	'*' (C_STAR)	other (C_OTHER)
CODE	SLASH / A_NONE	CODE / A_COPY	CODE / A_COPY
SLASH	SLASH / A_COPY	BLOCK / A_START	CODE / A_FLUSH
BLOCK	BLOCK / A_NONE	STAR / A_NONE	BLOCK / A_NONE
STAR	CODE / A_SPACE	STAR / A_NONE	BLOCK / A_NONE

A_NONE writes nothing, A_COPY writes the char, A_FLUSH writes '/' and the char, A_START counts one comment, A_SPACE writes one space. strip_text (given) does these outputs for you.

Do these steps:

- **Step 1:** TODO 1 (way 1) - next_switch: switch (s) with one case per row. In each case use if (c == C_SLASH) / if (c == C_STAR) and set the output and return the next state, e.g. if (c == C_SLASH) { *a = A_NONE; return SLASH; }
- **Step 2:** TODO 2 (way 2) - copy the SAME table into next_table (left parts) and act_table (right parts).
- **Step 3:** TODO 3 (way 2) - next_by_table: two lines, no switch and no if: *a = act_table[s][c]; return next_table[s][c];
- **Step 4:** in the terminal: cd q5_comments then make run - compare with expected_output.txt.
- **Step 5:** in the main folder: py tests\run_all_tests.py q5 (Mac: python3 tests/run_all_tests.py q5).

Input file code.txt:

```
/* BroBot speed check */
#include <stdio.h>

int main(void) {
    int laps = 12;      /* laps today */
    int avg = laps / 4; /* integer division */
    /* a comment
       on two lines */
    printf("%d\n", avg * 2); /** done **/
    return 0;
}
```

Expected output:

```
---- clean code ----

#include <stdio.h>

int main(void) {
    int laps = 12;
    int avg = laps / 4;

    printf("%d\n", avg * 2);
    return 0;
}
---- end ----
COMMENTS=5
UNCLOSED=NO
AGREE=YES
```

Reflection and Peer Learning Log (5 points)

Open reflection.txt and type a short answer after each colon (keep the labels). Help at least one classmate understand an idea - do not give them your code - and write it in the log.

Submit

```
git add q*/*.c reflection.txt
git commit -m "Exercise 10"
git push
```

Then check for the green check mark in the Actions tab on GitHub.

Grading

Question	Autograder tests	Points
Q1 Vending machine	given file 7, second file 5, table check 3	15
Q2 frfr	given file 7, second file 5, switch check 3	15
Q3 BroBank ATM	given file 8, second file 8, switch check 4	20
Q4 Door lock	given file 8, second file 7, table check 5	20
Q5 Remove the comments	given file 10, second file 8, both-ways check 7	25
Reflection	reflection.txt	5
Total		100

"second file" = the autograder also runs your program on a different input file, so do not hard-code the answers.

Cheat Sheet - switch vs 2D array

Every question in this lab starts the same way: DRAW the diagram, then turn it into C. Here is the tiny machine (output 1 when you read two 1s in a row) written both ways.

Step 1 - from the diagram to names (enum):

```
typedef enum { A, B, NUM_STATES } State;  /* A = 0, B = 1, NUM_STATES = 2       */
/* inputs:  the bit 0 or 1 is already a column number:  in = c - '0'             */
/* outputs: 0 or 1  (in other questions an enum: WAIT, VEND, ...)                */
/* arrows = NUM_STATES x NUM_INPUTS = 2 x 2 = 4 = the cells of each table        */
```

Arrow on the diagram	Way 1 (switch) writes	Way 2 (2D array) writes
A --0/0--> A	case A: if (in == 0) { out = 0; next = A; }	next_state[A][0] = A out_table[A][0] = 0
A --1/0--> B	case A: else { out = 0; next = B; }	next_state[A][1] = B out_table[A][1] = 0
B --0/0--> A	case B: if (in == 0) { out = 0; next = A; }	next_state[B][0] = A out_table[B][0] = 0
B --1/1--> B	case B: else { out = 1; next = B; }	next_state[B][1] = B out_table[B][1] = 1

Way 1 - switch: one case per STATE, an if picks the ARROW

```
int step_switch(State *s, int in) {
    int out;
    switch (*s) {                          /* 1. which circle am I in?      */
    case A:
        if (in == 0) { out = 0; *s = A; }  /* 2. which arrow leaves it?     */
        else         { out = 0; *s = B; }
        break;                             /* 3. never forget the break     */
    case B:
        if (in == 0) { out = 0; *s = A; }
        else         { out = 1; *s = B; }
        break;
    default:                               /* safety: unknown state         */
        out = 0; *s = A;
        break;
    }
    return out;
}
```

Way 2 - 2D arrays: row = current state, column = input

```
/*                                      in 0  in 1 */
const State next_state[NUM_STATES][2] = { {A, B},      /* row A */
                                          {A, B} };    /* row B */
const int   out_table[NUM_STATES][2]  = { {0, 0},      /* row A */
                                          {0, 1} };    /* row B */

int step_table(State *s, int in) {
    int out = out_table[*s][in];           /* look up the output (OLD state) */
    *s = next_state[*s][in];               /* look up the next state         */
    return out;                            /* no switch, no if!              */
}
```

The Mealy loop - the same order in EVERY question:

Step	Code	Why
1. read the input	col = column_of(c);	turn the char into a column number
2. remember the state	old = s;	you print it, and it decides the output

3. output	out = out_table[s][col];	the output belongs to the ARROW (input/output)
4. move LAST	s = next_state[s][col];	follow the arrow to the next state

```
State s = START;                                /* the start arrow                    */
while ((c = fgetc(fp)) != EOF) {                /* or: for each char of a line        */
    int col = column_of(c);                     /* char -> column number              */
    if (col < 0) { /* SKIP / ERR */ continue; } /* never index a table with -1!       */
    State old = s;
    int out = step(&s, col);                    /* way 1 or way 2: same result        */
    printf("STEP %d: %s --%c/%d--> %s\n", k, NAME[old], c, out, NAME[s]);
}
```

Which way should I use?

	switch	2D array
the rules live in	the code (one case per state)	data (two tables)
easy when	few states, rules with extra data or conditions (BroBank: the balance)	many states and inputs, a full diagram (vending, the door lock)
to add a state	add a case	add a row to EVERY table
typical bug	missing break / missing case	row and column swapped, wrong cell
check it	every case has an if for every input	rows = NUM_STATES, columns = NUM_INPUTS

Five rules that prevent most bugs:

- The enum order and the names array order must match: enum { S0, S5, S10 } -> NAME[] = {"S0", "S5", "S10"}.
- A column number must be checked before it is used: column_of() returns -1 for a char that is not an input.
- Save old = s, read the output with the OLD state, and move LAST.
- Every switch case ends with break (or return). Add default: so -Wall does not warn about a missing state.
- Trace on paper: write one row per input (state, input, output, next) before you trust the program.

Important: Both ways must give EXACTLY the same output. In Q5 your program prints AGREE=YES only if they do.